第二阶段：vdot.vv 设计文档

工大喵喵队 2026 年 11 月

一、指令语义

vdot.vv 从两个向量寄存器各取 8 个有符号 int8，做点积，结果写到 vd 低 32 位，高 96 位清零：

```
vd[31:0]   = Σ(i=0..7) sext8(vs1.byte[i]) × sext8(vs2.byte[i])
vd[127:32] = 0
```

sext8 把 8 位字节解释成有符号 int8：0–127 不变，128–255 减 256。

合法配置只有一种：

参数	要求
VL	8
SEW	8
LMUL	1
vstart	0
vm	1
VLEN	128

不满足时抛非法指令异常（mcause=2），vd 保持原值。mtval 分两类：VL 类在执行阶段发现，mtval=0；SEW/LMUL/vstart 类在译码阶段发现，mtval=指令字 0x0220a18b。指令只处理低 8 个字节，高 64 位不参与运算。

二、指令编码

字段	位宽	位置	值
funct7	7	31:25	0000001
rs2	5	24:20	vs2
rs1	5	19:15	vs1
funct3	3	14:12	010
rd	5	11:7	vd
opcode	7	6:0	0001011（custom-0）

示例：

```
vdot.vv v3, v1, v2
funct7=0000001  rs2=00010(v2)  rs1=00001(v1)
funct3=010      rd=00011(v3)   opcode=0001011
= 0x0220A18B
```

汇编写法是 `vdot.vv vd, vs1, vs2，vs1=rs1、vs2=rs2`。标准 RVV 的顺序是 `vd, vs2, vs1`，和我们的约定不一样；点积对两个源操作数对称，交换 `vs1/vs2` 不改变结果。

三、微架构设计

我们没有另起一条流水线，而是复用香山已有的向量译码、重命名、分派、发射、ROB、写回和异常通道。

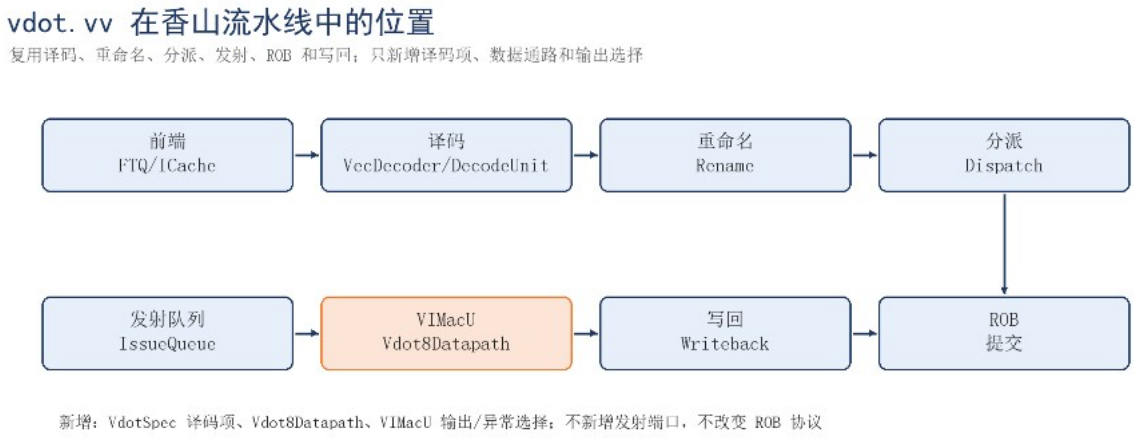


图 1 vdot.vv 在香山流水线里的位置

改 4 个已有文件，新增 1 个文件：

文件	改动	作用
`decode/VecDecoder.scala`	加一行 BitPat	识别成 `FuType.vimac`、`fuOp=0x100`、`VEC_VVV`
`decode/DecodeUnit.scala`	加非法配置判断	`vtype.illegal`、`vsew≠0`、`vlmul≠0`、`vstart≠0` 在译码阶段报非法
`decode/isa/bitfield/RiscvInst.scala`	放宽 `isVecArith`	让自定义 opcode 进入向量算术路径
`fu/wrapper/VIMacU.scala`	实例化数据通路、加输出/异常选择	复用 VIMac 发射口和写回
`fu/wrapper/Vdot8Datapath.scala`	新增	8 路乘加树、两级寄存器

VdotSpec：

```
object VdotSpec {
  val instruction = BitPat("b0000001??????????010?????0001011")
  val fuOp = BitPat("b100000000")
  def isDot(op: UInt): Bool = op === "h100".U
}
```

VecDecoder 里加：

```
xiangshan.backend.fu.wrapper.VdotSpec.instruction ->
  OPMVV(T, FuType.vimac, xiangshan.backend.fu.wrapper.VdotSpec.fuOp,
        F, T, F, UopSplitType.VEC_VVV)
```

DecodeUnit 里加：

```
isAmocasQIllegal ||
(xiangshan.backend.fu.wrapper.VdotSpec.instruction === inst.ALL &&
  (io.enq.vtype.illegal || io.enq.vtype.vsew =/= 0.U ||
   io.enq.vtype.vlmul =/= 0.U || io.enq.vstart =/= 0.U))
```

这样 vdot 在译码后就是一个正常的向量 uop，有 pdest、robIdx、fuType、fuOpType，后面的重命名、分派、发射、写回、提交都不用为它单独开路径。

四、数据通路

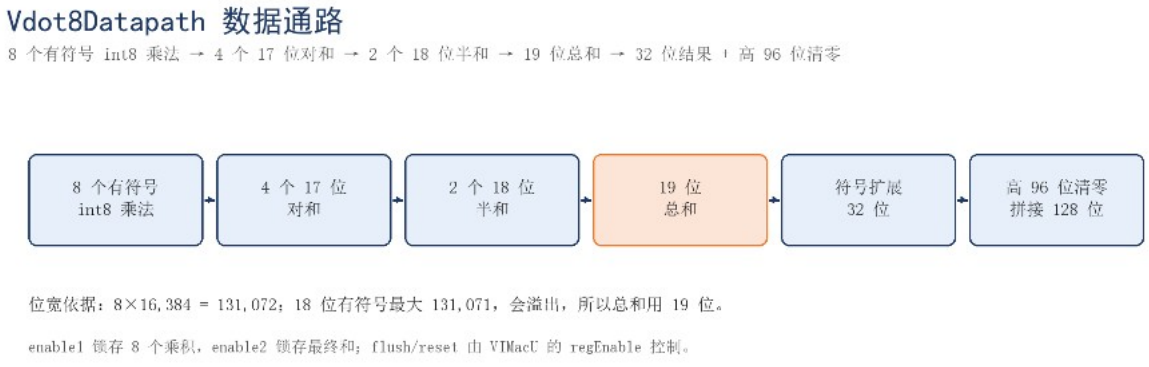


图 2 Vdot8Datapath 数据通路

数据通路分两级：

1.8 个有符号 8×8 乘法器，乘积锁存在 8 个 16 位寄存器里； 2.4 个 17 位对和 → 2 个 18 位半和 → 1 个 19 位总和 → 符号扩展成 32 位 → 拼 96 位 0。

位宽计算：

量	最小值	最大值
单个乘积	−16,256	16,384

量	最小值	最大值
8 项和	−130,048	131,072
19 位有符号	−262,144	262,143

8×16,384 = 131,072，18 位有符号最大 131,071，会溢出，所以总和用 19 位。

核心代码：

```
class Vdot8Datapath extends Module {
  val io = IO(new Bundle {
    val vs1 = Input(UInt(128.W))
    val vs2 = Input(UInt(128.W))
    val enable1 = Input(Bool())
    val enable2 = Input(Bool())
    val vd = Output(UInt(128.W))
  })
  val products = (0 until 8).map { i =>
    RegEnable(io.vs1(8*i+7,8*i).asSInt * io.vs2(8*i+7,8*i).asSInt, io.enable1)
  }
  val pairs  = products.grouped(2).map(x => x(0) +& x(1)).toSeq
  val halves = pairs.grouped(2).map(x => x(0) +& x(1)).toSeq
  val sum    = halves(0) +& halves(1)
  val result = RegEnable(sum.pad(32).asUInt, io.enable2)
  io.vd := Cat(0.U(96.W), result)
}
```

`enable1` 锁存 8 个乘积，`enable2` 锁存最终和；旧事务不会带到新路径。

VIMacU 里接：

```
private val isDot = VdotSpec.isDot(fuOpType)
private val outIsDot = VdotSpec.isDot(outCtrl.fuOpType)
private val dot = Module(new Vdot8Datapath)
require(cfg.destDataBits == 128 && latency == 2)
dot.io.vs1 := vs1
dot.io.vs2 := vs2
dot.io.enable1 := regEnable(1) && isDot
dot.io.enable2 := regEnable(2)

mod.io.fire := io.in.valid && !isDot
...
io.out.bits.res.data := Mux(outIsDot, dot.io.vd, mgu.io.out.vd)
io.out.bits.res.vxsat.get := !outIsDot && (outVxsat & mgu.io.out.active).orR
```

`mod.io.fire` 加`!isDot`，让原来的 MGU/掩码路径不处理 vdot；`Mux` 只在 vdot 时选点积结果；`vxsat` 对 vdot 清零。

五、异常与别名

执行阶段检查：

```
val dotIllegal = outVecCtrl.vill || outVecCtrl.vsew =/= 0.U ||
  outVecCtrl.vlmul =/= 0.U || outVl =/= 8.U || outVstart =/= 0.U || !outVm
io.out.bits.ctrl.exceptionVec.get(ExceptionNO.illegalInstr) :=
  Mux(outIsDot, dotIllegal, mgu.io.out.illegal)
```

译码阶段也挡了一层 vtype/vstart 非法。异常语义：

· mcause=2；

· mepc 指向 vdot；

· vd 不变；

· mret 后能继续执行；

· mtval 按发现阶段分两类。

mtval 实测：

配置	mcause	mtval	发现阶段
VL=0/1/7/9	2	0x0	执行阶段（VIMacU dotIllegal）
SEW=16/LMUL=2/vstart=1	2	0x0220a18b	译码阶段（DecodeUnit，TrapInstMod 填指令字）
标准非法 0x7b	2	0x7b	常规非法指令

8 种非法配置由 software/mtval_probe.c 逐例打印，证据是 results/t2-mtval-probe/mtval-probe.log。这次是--no-diff 直接打印，不是 difftest；带 difftest 的 A/B 只对照了 SEW=16。

software/exception_check.S 的 a0 版做了三次 trap：VL=0、标准非法 0x0000007B、再次 VL=0。每次都检查 mcause=2、mepc 指向对应指令、vdot 的 mtval=0、标准非法指令的 mtval=0x7B、v3 保持 0xFF、mret 后能继续执行。日

志是 results/t5b-clean-ref/exception-check-a0.log，274 条指令、2,865 周期，a0=0。

别名方面，vd 可以和 vs1、vs2 或 v0 重叠。NEMU helper 在写 vd 前先读完 8 对源字节，所以 vdot.vv v1,v1,v2、vdot.vv v2,v1,v2、vdot.vv v0,v1,v2 都是先读后写。三个变体各跑 1,030 个用例，包含在 a0 回归中，a0=0。

六、RTL 文件与验证

文件	说明
rtl/xiangshan/Vdot8Datapath.scala	数据通路和 VdotSpec
rtl/xiangshan/xiangshan-vdot-complete.patch	4 改 1 增的完整补丁
rtl/standalone/vdot8.sv	独立 SV 原型
results/t5b-clean-ref/vdot-a0.log	a0 版整机回归
results/t5b-clean-ref/exception-check-a0.log	a0 版异常测试
results/t2-mtval-probe/mtval-probe.log	8 种非法配置的 mtval 实测

a0 版把 trap 字的 rs1 改成 a0（.word 0x0005006b），所以 GOOD TRAP 等价于 a0=0。历史日志仍是 x0 版本，只能说明运行至结束、无 mismatch。

验证结果：

层级	内容	结果
RTL 单元	69,656 个 oracle 向量，69,657 个输出，4 个取消事务	独立 testbench 输出 PASS
香山整机 a0 版	1,030 个用例 × 5 种实现变体	GOOD TRAP，a0=0，841,260 instr，790,553 cycles
正/负/边界	定向 + 随机 + 全字节对	包含在 a0 回归中，a0=0
别名	vd=vs1、vd=vs2、vd=v0	包含在 a0 回归中，a0=0
异常 a0 版	VL=0、标准非法、再次 VL=0	GOOD TRAP，a0=0，274 instr，2,865 cycles

层级	内容	结果
非法配置	8 种，VL 类/译码类/标准非法	mtval 两类，逐例打印（`--no-diff`）
高位独立性	4 组高半图案	程序输出 `PASS`，`echo16=ok`、`low=-168`、`upper96=0,0,0`
背靠背	三条紧邻 vdot	程序输出 `PASS`，结果−168/204/−4

高位/背靠背的日志在 `results/t3-highbits-backtoback-v2/`，4 组高半图案和 3 条背靠背都带`[RESULT] PASS`，exit 0；它们不是 a0 回归。

七、扩展与限制

结构上可以扩：

目标	需要改什么
SEW=16	8 个 16×16 乘法器，乘积 32 位，和需要 35 位左右
SEW=32	8 个 32×32 乘法器，乘积 64 位，和需要 67 位左右
VLEN=256	16 个元素，乘法器/加法树翻倍，或者分两拍复用

参数化原型只做到独立模块级：n=4/8/16 生成 Verilog，iverilog 跑 471/479/449 个输出，0 错误，3 处变异都被抓到。证据在 `results/parametric-verification/parametric-verification.json`。它没有接进香山，控制/位宽和正式 `Vdot8Datapath` 不同，只证明数据通路；没有综合、STA，不能据此说时序更好。Chisel 源码里是平衡树，但 firtool 输出是左结合加法链，时序效果要综合/STA 确认。

当前限制：

· 只支持 VL=8、SEW=8、LMUL=1、vstart=0、vm=1；

· 异常覆盖 M 模式 8 种非法配置；S/VS 委托、RNMI、NMIE=0 早退路径未覆盖；

· 参数化只到独立模块级，未做整核集成；没有综合、STA、面积、频率、功耗数据；

· vd 高 96 位清零是按赛题语义做的，没有做“保留旧值”的兼容模式；

· 没有形式化验证，69,656 向量和 1,030 用例是实测覆盖，不是数学证明。